Lightning Network
Payment-Channel Engine

Commitment updates, HTLCs and penalty enforcement

Rishabh Yadav (230859)
Arnav Gupta (240189)

CS731

Bitcoin Settlement Has Scarce Capacity

01 Bitcoin targets an average block interval of about 10 minutes. Confirmation time varies, and finality is probabilistic.

02 SegWit limits a block to 4,000,000 weight units. Transactions per second depend on transaction size and witness composition.

03 Every on-chain payment competes for block space. During high demand, fees can overwhelm a small payment.

04 Motivating examples include streaming payments and metered API access. A $3 transaction fee makes a $0.10 payment impractical.

UTXOs: Inputs Are Consumed, Outputs Are Created

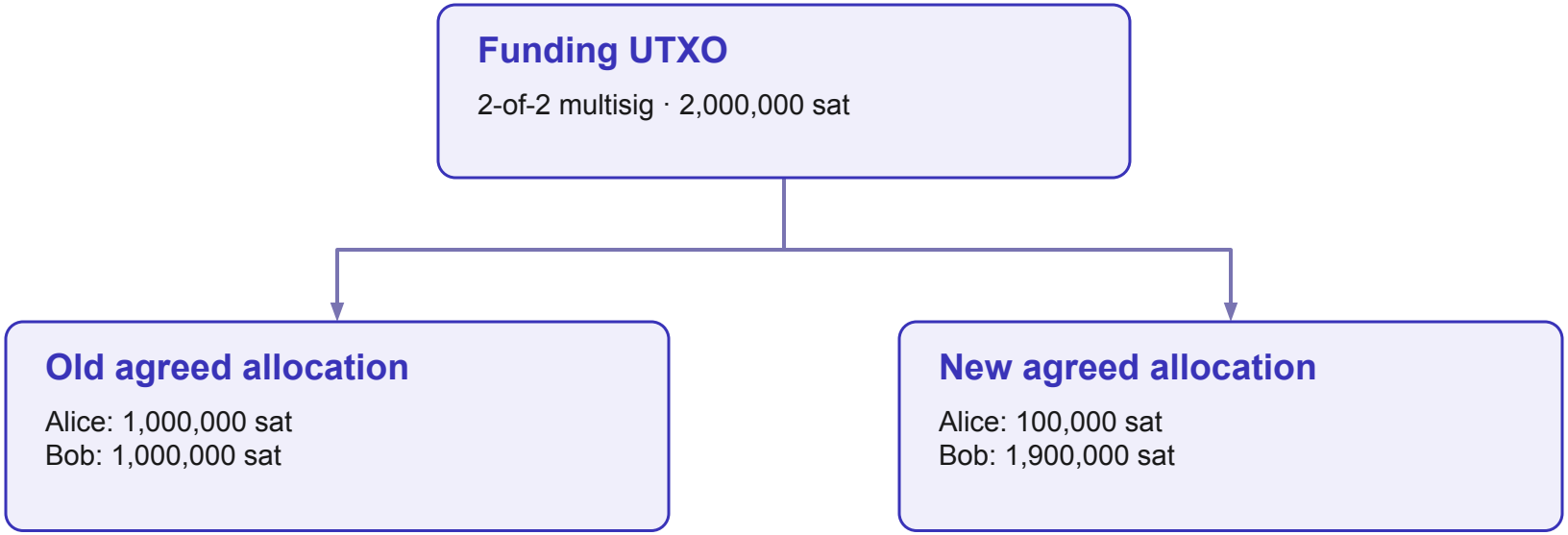
01 Funds reside in transaction outputs protected by spending conditions. An outpoint identifies an output as txid:vout.

02 A transaction consumes unspent outputs and creates new ones. A signature alone does not consume a UTXO.

03 Two commitments may spend the same funding output. Either may be valid before one confirms, but both cannot settle together.

04 A newer private agreement does not invalidate an older signature. An account-based contract also needs explicit rules to enforce off-chain state ordering.

The Cheating Dilemma: Old Signatures Survive



Both commitments spend the same UTXO. Either commitment can spend that output while it remains unspent.

Approaches to Repeated Payments

Approach	Mechanism	Relevant tradeoff
Custodial balances	Internal ledger	Custodian controls redemption
Spilman channels	One-way updates, refund deadline	One-way payment capacity
Decker–Wattenhofer	Duplex invalidation trees	More complex transaction structure
Poon–Dryja	Asymmetric penalty commitments	Monitoring and settlement deadlines

Our implementation follows a restricted Poon–Dryja / BOLT transaction profile. The white paper is foundational design work, not a peer-reviewed paper.

Contribution: the Channel Mechanism Is Implemented Here

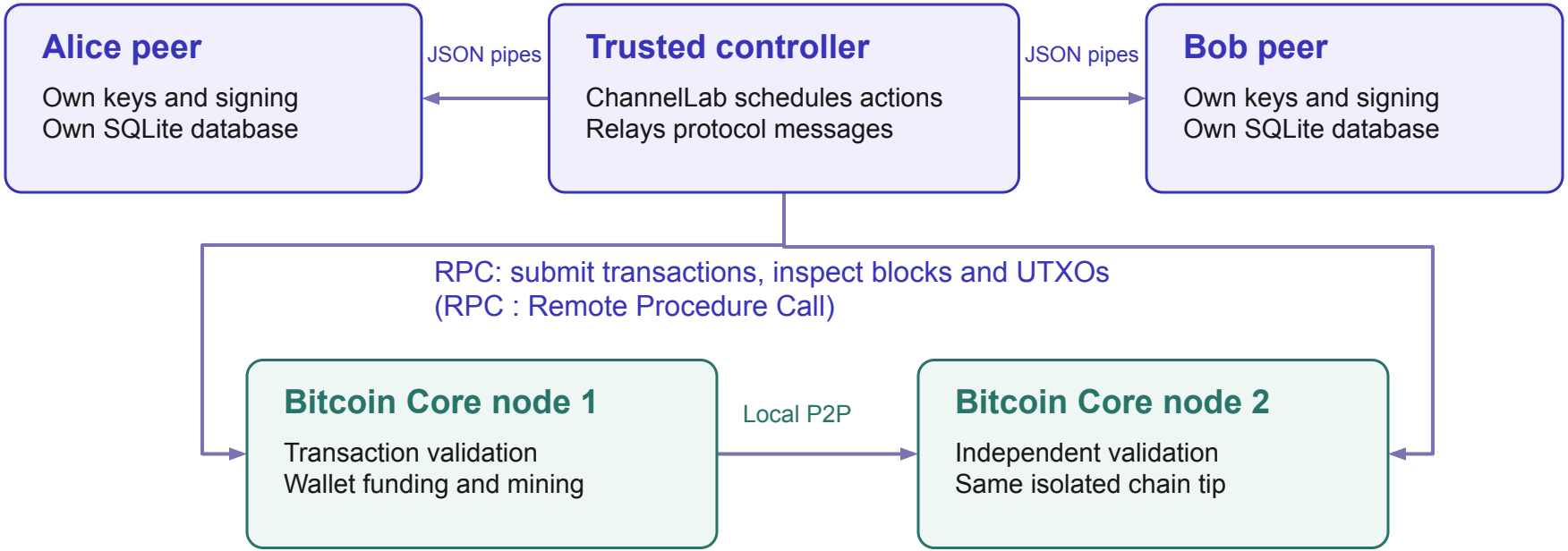
01 Python constructs scripts, transaction serialization, BIP 143 digests, commitments and revocation key combinations.

02 Coincurve supplies secp256k1 operations, ECDSA signing and verification. It does not implement the channel.

03 Bitcoin Core validates transactions and relay policy. Its wallet constructs the funding transaction using our funding script.

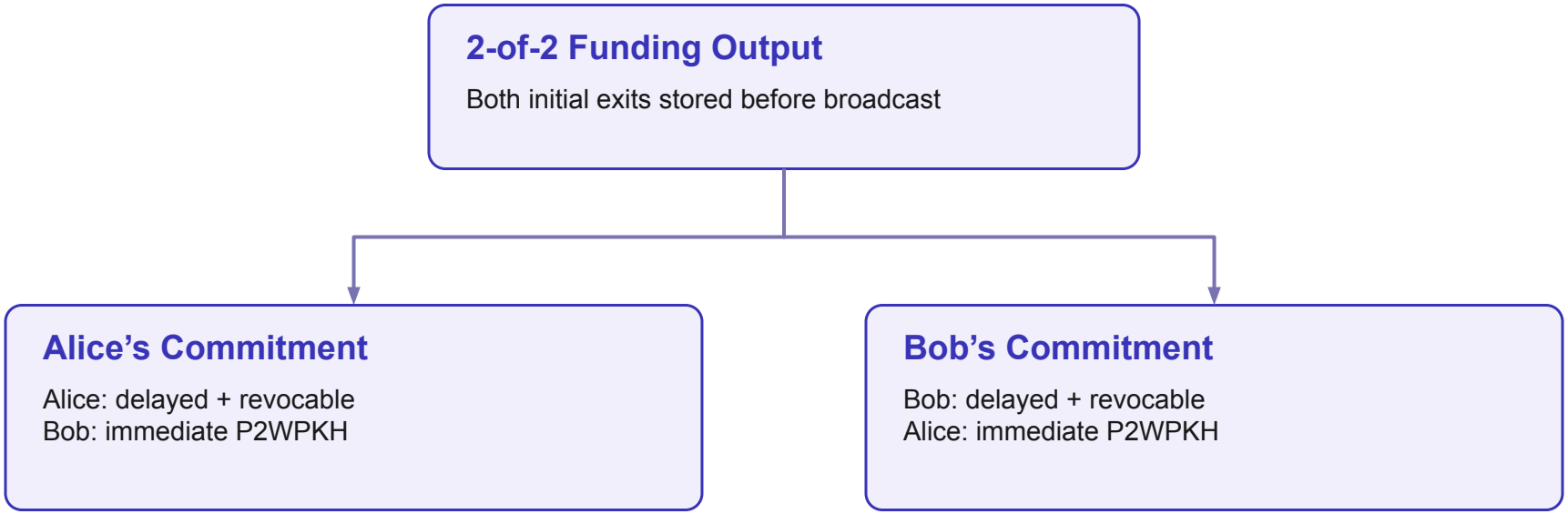
04 No LND or Core Lightning engine implements the transitions. The result is a restricted educational protocol, not full BOLT wire compatibility.

Architecture: Separate Peers, Shared Test Chain



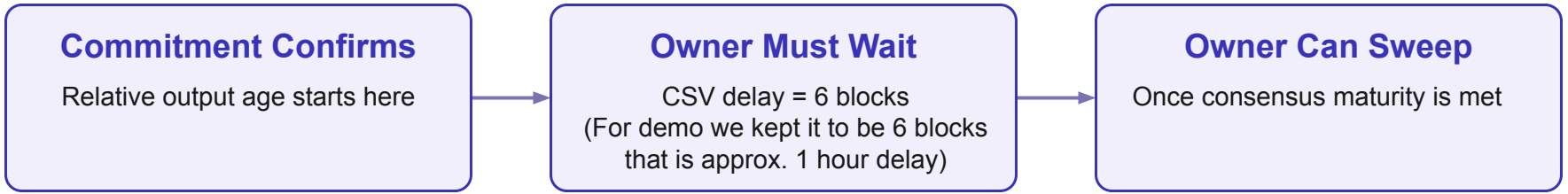
Peers construct and sign channel transactions. Core checks them on regtest or private OP_TRUE signet.

Funding and Asymmetric Commitments



The broadcaster waits for CSV (CheckSequenceVerify) maturity. A disclosed revocation secret gives the counterparty an immediate penalty path.

Six-Block CSV Delay: Time to Detect a Breach



Revoked State: Counterparty Can Use the Immediate Revocation Branch

The delay gives the honest peer time to observe the breach and confirm a penalty before the owner spends. It delays the broadcaster’s output, not each off-chain payment.

Why Six Blocks? a Demo Parameter, Not a Safety Guarantee

Purpose of the Delay

Protect the response window.
Trade faster owner withdrawal for more monitoring time.

Why This Demo Uses Six

Keep isolated scenarios short.
Core enforces the lock on-chain.
Not a production recommendation, just for testing purposes.

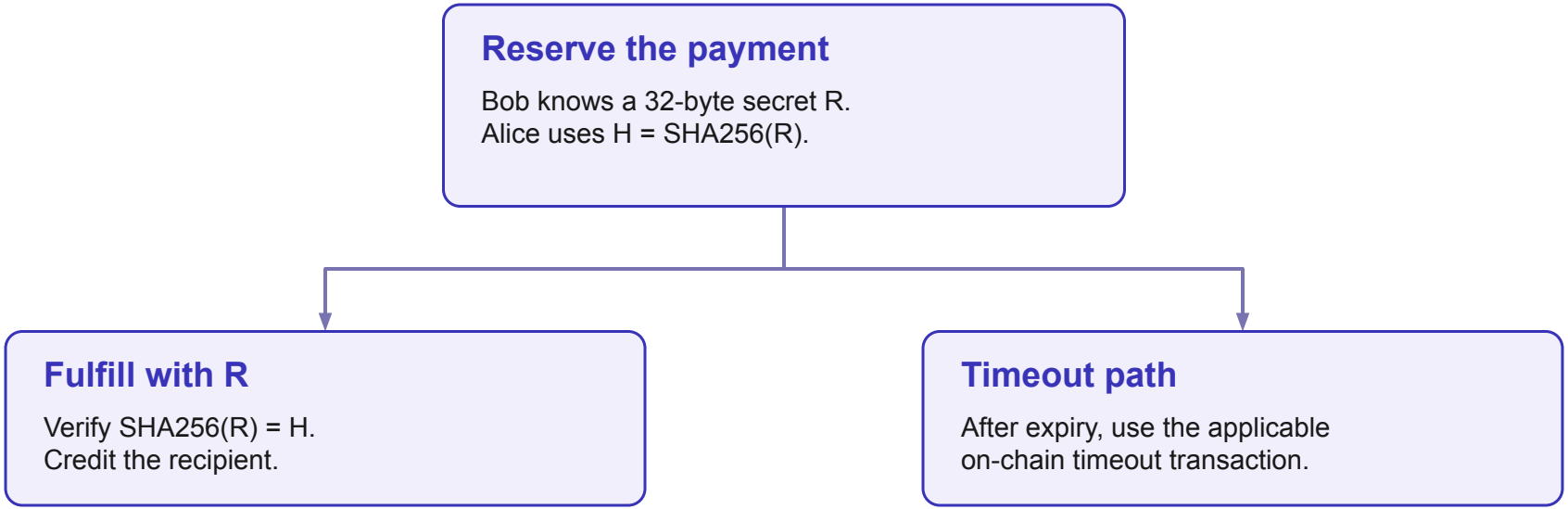
Verified Behavior

Honest close: early owner spend is rejected; a later spend succeeds.
Missed window: confirmed owner withdrawal makes the late penalty fail.

Six blocks is not a fixed wall-clock hour. Private test blocks are mined by the harness.

HTLC: a Payment Conditional on a Secret

Hash Time-Locked Contract



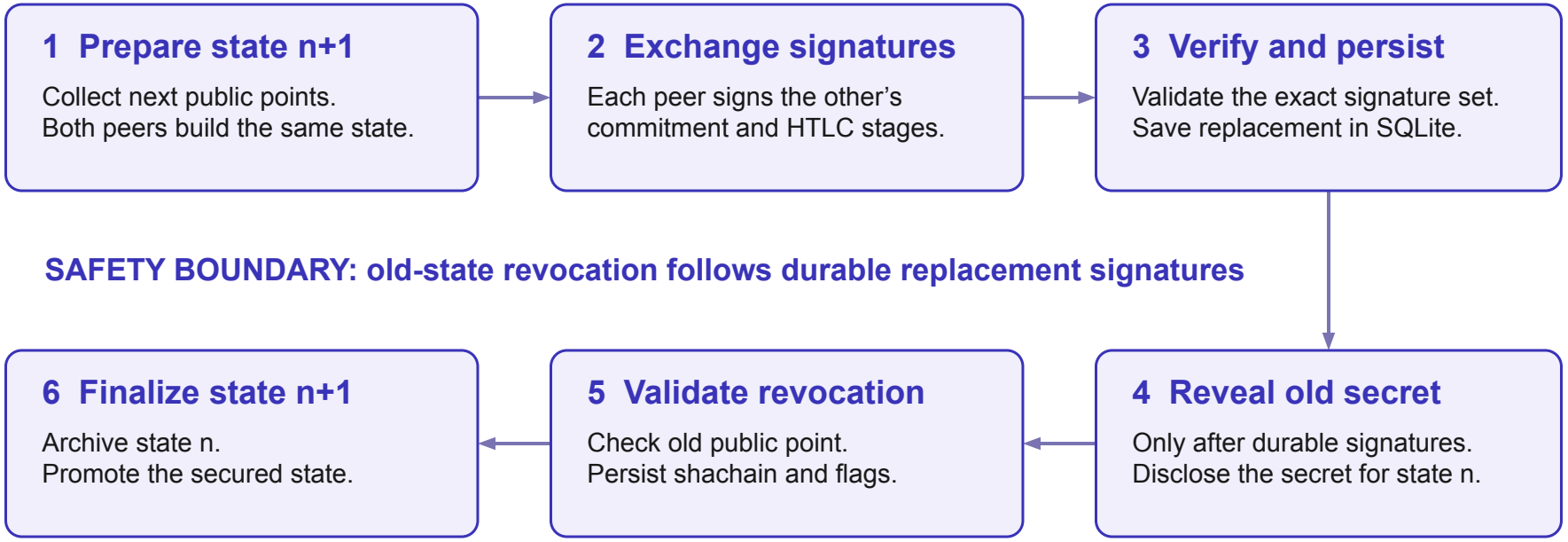
Off-chain fail is a separate agreed update that refunds the sender. Expiry does not automatically erase the preimage path.

A Simple Payment Example

Stage	Alice (sat)	Pending HTLC (sat)	Bob (sat)
Initial allocation	1,000,000	0	1,000,000
Add payment	950,000	50,000	1,000,000
Fulfill payment	950,000	0	1,050,000
Alternative: fail	1,000,000	0	1,000,000

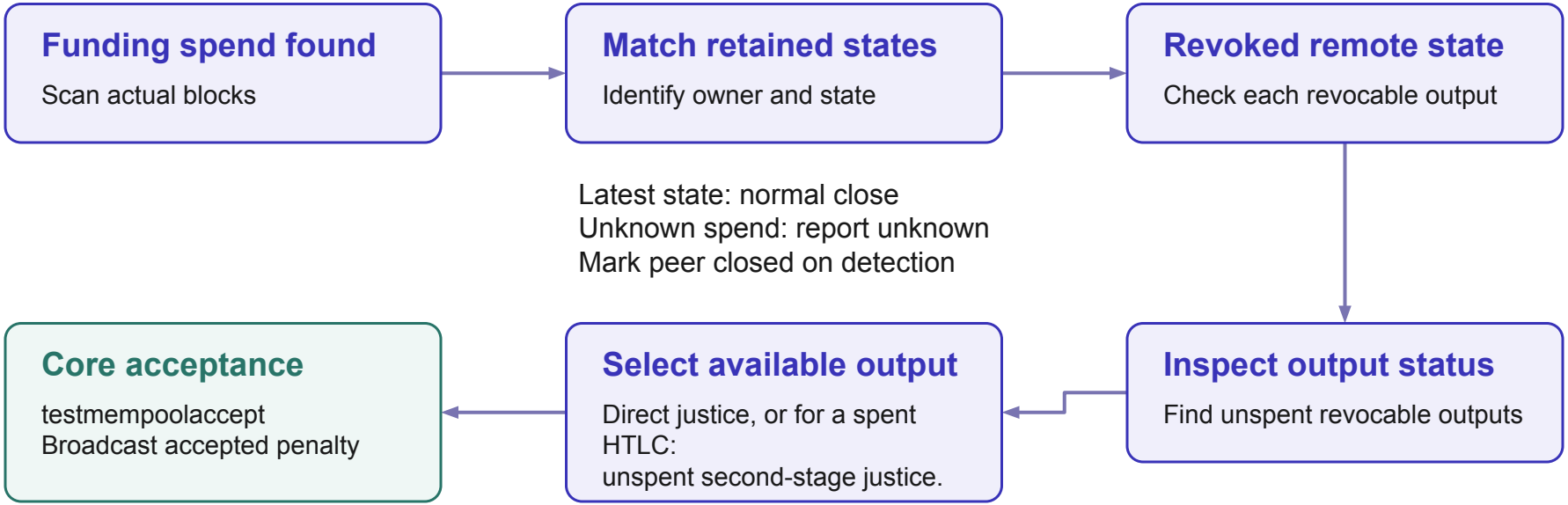
Illustrative 2,000,000 sat channel from the report. Allocations precede settlement fees. A failed payment returns the reserved amount to its sender.

Channel Update: Secure the Replacement First



Controller serializes both peers' steps. A pending update blocks commitment publication until reconciled.

Penalty Flow: Detect, Classify, Recover



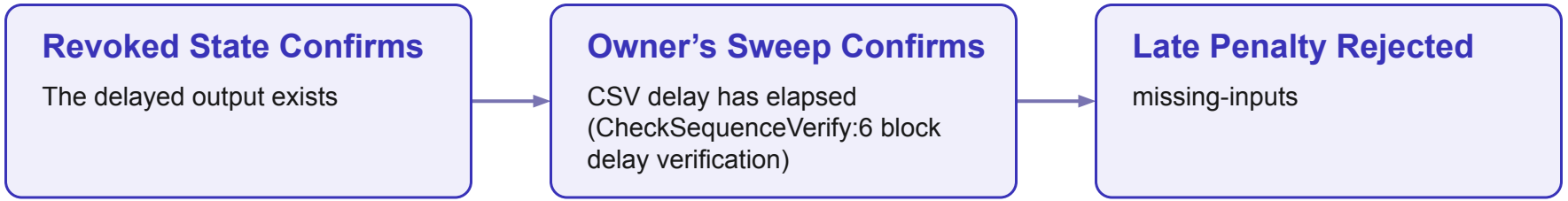
Only available outputs can be recovered. A rejected penalty raises an error; a spent output is not rescued.

Breach Example: Recovery After Settlement Fees

Recorded event	Observation
Latest state	State 2 before fees: Alice 950,000 / Bob 1,050,000 sat
Alice broadcasts state 0	Revoked commitment confirms at block 135
Justice transaction	Penalty confirms at block 136
Bob’s recovered outputs	995,380 + 999,000 = 1,994,380 sat
Settlement fees	3,620 + 1,000 + 1,000 = 5,620 sat

The penalty confirmed one block after the revoked state. Funding fees are outside the channel capacity.

Missed Response Window: a Late Penalty Fails



A valid revocation key cannot recover an output that has already been spent.

Timely monitoring and on-chain response remain necessary.

Pinning Risk and Fee Inflation

RECORDED BITCOIN CORE 28.1 TEST

- **Alice closes with a payment pending.** Bob can claim the HTLC. After expiry, Alice's timeout competes to spend the same output.
- **First check.** Bob's claim is unconfirmed and pays a 500-sat fee. Core says it would accept Alice's 3,315-sat timeout in its place.
- **A follow-on payment appears.** It spends the payout from Bob's claim, so replacing the claim would remove both transactions from the mempool.
- **Core rejects the same timeout.** The two transactions it would remove pay 10,500 sats in total. Alice's 3,315-sat fee is now too low.

Alice's fee was already signed. Core checked acceptance without broadcasting the timeout.

FEE INFLATION (HYPOTHETICAL EXAMPLE)

When Bitcoin fee rates rise, a presigned refund can become too cheap to confirm promptly.

If timely confirmation would now take about 20,000 sats in fees, Alice's timeout still pays 3,315 sats.

Use Cases and Settlement Lessons

- **Pay per use.** An API could charge per request, or a stream could charge per minute. Many off-chain payments can share the Bitcoin fees for opening and closing a channel.
- **Keep a way to exit.** After a completed update, either peer can close without further cooperation. Interrupted updates currently require reconciliation or manual recovery.
- **Recover in time.** In our tests, the penalty worked when it confirmed before the delayed payout was spent. The late penalty failed after that payout was gone.

Limitations and Security Assumptions

01 Educational prototype tested on regtest and private signet. One channel, one pending HTLC at a time, whole satoshis, and a fixed fee rate.

02 Trusted local controller and operating system. Test databases retain plaintext secrets. No hostile-host protection is claimed.

03 No Lightning wire transport, routing, cooperative close or external-client interoperability.

First Priority: Recovery and Continuous Monitoring

- **Recover interrupted updates.** If Alice has finalized an update that Bob still has pending, Bob can finish it. If only Alice saved the proposal, the engine stops for manual recovery. Test crashes at each step so peers can resume from a safe state.

- **Keep monitoring after a restart.** Save the last block checked. If Bitcoin replaces blocks, recheck the channel close and resubmit any valid recovery transaction that is still needed.

- **Add an external watchtower.** Our lab runs a local resolver on demand. A separate service could watch Bitcoin and use encrypted recovery data to submit a penalty while a peer is offline.

Future Experiments for the Channel Engine

- **Test more flexible settlement fees.** Compare a fee-bumping design with our fixed-fee engine. Measure Core acceptance, total fees, and whether settlement confirms in time.
- **Warn when a pending payment may lose its on-chain payout.** Use our fee model to flag HTLCs that a commitment would omit. Test how many small payments the warning would affect.
- **Test beyond the local lab.** Run our two peers on public signet to observe confirmation timing. Later, implement standard Lightning messages and test with an external client.

Conclusion

- **We built an educational Bitcoin payment-channel engine.** Two peers update balances off-chain. Bitcoin Core validated the funding, closing, and penalty transactions on regtest and private signet.
- **We demonstrated how revocation is enforced.** Peers save their replacement signatures before revoking an old state. In the breach demo, Bob's penalty confirmed one block after Alice's revoked close.
- **Recovery depends on timing and fees.** A late penalty failed after Alice's withdrawal confirmed. In a separate test, Core's replacement policy rejected a presigned HTLC timeout.

Research References and Policy Sources

01 Poon & Dryja (2016), The Bitcoin Lightning Network. Foundational whitepaper. BOLT 2 / 3 / 5 define the pinned protocol reference.

02 Harris & Zohar, Flood & Loot, ACM AFT 2020. Mizrahi & Zohar, Congestion Attacks in Payment Channel Networks, FC 2021.

03 Decker & Wattenhofer, A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels (2015).

04 BIPs 68 and 112 (CSV), 141 and 143 (SegWit), 325 (signet), and 431 (relay topology).

Thank you

Questions?

Our Github Repo:
https://github.com/rishabh2625/CS731-Blockchain-Project-Lightning-Channel-

Appendix

Validation results and implementation details

Recorded Validation Coverage

Evidence	Recorded result
Unit / vector suite	30 passed, 1 optional integration test skipped
Core scenario runs	12 regtest and 12 private signet passed
Honest and revoked-state closes	Both Alice and Bob directions
HTLC resolution	Owner, direct, revoked and second-stage paths
Adverse scenarios	Missed response window and fee obstruction

Adverse cases pass when the expected rejection occurs. Two Core 28.1 nodes validated the private OP_TRUE signet runs.

Dual-key Revocation: Both Secrets Are Necessary

01 Let B = bG be Bob's revocation basepoint, and P = pG Alice's per-commitment point. Define h1 = SHA256(B || P), h2 = SHA256(P || B).

02 Public revocation key: R = h1·B + h2·P. Both peers can calculate it before disclosure.

03 Private revocation key: r = (h1·b + h2·p) mod n. Bob learns p only when Alice revokes that state.

04 Before disclosure neither peer has both private terms. Afterwards Bob can spend Alice's revocable output. The payment preimage is a different secret.

Commitment Construction and Fees

01 Transaction bytes and BIP 143 SIGHASH_ALL digests are built directly. Coincurve signs the resulting digests.

02 Outputs follow deterministic ordering. The funder pays the commitment base fee.

03 Base fee = floor((724 + 172 × untrimmed HTLCs) × feerate_per_kw / 1000).

04 Dust and uneconomic HTLC outputs are trimmed. Their omitted value increases the actual transaction fee.

HTLC Resolution Paths

HTLC relative to broadcaster	Broadcaster's path	Counterparty's path
Offered HTLC	Timeout, then delayed sweep	Direct preimage spend
Received HTLC	Success, then delayed sweep	Direct timeout spend
Revoked HTLC	Original paths remain valid	Immediate revocation spend
Revoked second stage	Delayed owner sweep	Second-stage penalty

Second-stage signatures are collected during the state update. Settlement does not require the remote peer to return online.

Local Off-Chain Payment Latency



50 payments and 100 durable updates per run. Includes add + fulfill; excludes block confirmation and WAN latency.

Recorded Transaction Costs

Transaction example	vB	Fee (sat)	sat/vB
Funding (breach_alice)	165	330	2.00
Commitment, no HTLC	181	3,620	20.00
Commitment, one HTLC	224	4,480	20.00
Main justice (breach_alice)	121	1,000	8.26
Fixed HTLC timeout probe	165	3,315	20.09

The selected commitments differ by 43 vB. BOLT fee weights are construction estimates; actual serialization depends on signatures and output trimming.

PTLCs and Payment Privacy

- **Point locks.** A point time-locked contract (PTLC) ties payment to an elliptic-curve point. The corresponding secret can complete an adaptor signature.
- **Routed-payment privacy.** Different locking points along a route can make it harder to link its individual payment hops.
- **On-chain privacy and implementation.** Some Taproot key-path spends keep fallback scripts hidden. A PTLC design would change key handling, signing, and recovery.